SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I G Venkata Ganesh-192425381 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: G V Ganesh

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

ASSESSMENT TOOL-1

Name : G Venkata Ganesh

Reg.no : 192425381

Course : Computer Architecture

Course Code : CSA1206

Department : B.Tech-AICML

APPLICATION BASED QUESTIONS

1) Analysis of Smart City Surveillance System Performance

A smart city surveillance system continuously receives video streams from multiple cameras. These video feeds must be transferred from **I/O devices** (cameras) to **memory**, processed by the **CPU**, and then stored or displayed. During peak hours, many cameras send data simultaneously, causing heavy traffic between the CPU, memory, and I/O devices, which results in lag and delayed threat detection.

Interaction between CPU, Memory, and I/O

CPU (Central Processing Unit)

- Processes video frames and executes threat detection algorithms.
- Requires a continuous supply of data from memory.
- If data arrives slowly, the CPU remains idle, reducing system performance.

Memory (RAM)

- Temporarily stores incoming video frames.
- Acts as a buffer between I/O devices and the CPU.
- If memory becomes overloaded due to multiple video streams, access time increases, causing delays.

I/O Devices

- Cameras continuously generate large amounts of video data.
- Data is transferred to memory through the system bus.
- When many cameras transmit simultaneously, I/O bandwidth becomes congested.

Cause of Lag

- Multiple cameras generate huge data simultaneously.
- Memory becomes busy storing and retrieving video frames.
- CPU waits for data because of slower memory and I/O transfers.
- As a result, threat detection is delayed.

Effect of Bus Structures

Single-Bus Structure	Multi-Bus Structure
CPU, memory, and I/O share one common bus.	Separate buses are used for CPU-memory and I/O communication.
Only one device can use the bus at a time.	Multiple data transfers occur simultaneously.
High bus congestion during peak traffic.	Reduced congestion and higher bandwidth.
Slower data transfer and increased waiting time.	Faster communication and improved performance.
Suitable for small systems.	Suitable for high-performance surveillance systems.

Program

```
START
    Read video frames from camera
    WHILE video is available
        Transfer frame from I/O device to Memory
        CPU fetches frame from Memory
        Process frame for threat detection
        IF threat detected THEN
            Display "Threat Detected"
        ELSE
            Display "No Threat"
        ENDIF
    ENDWHILE
STOP
```

OUTPUT

Improvements in the Instruction Execution Cycle

The instruction execution cycle consists of:

1. **Fetch** – Retrieve instruction from memory.
2. **Decode** – Interpret the instruction.
3. **Execute** – Perform the required operation.
4. **Store (Write Back)** – Save the result.

Performance can be improved by:

- **Pipelining:** Executes multiple instructions simultaneously by overlapping fetch, decode, and execute stages.
- **Cache Memory:** Stores frequently accessed instructions and data, reducing memory access time.
- **DMA (Direct Memory Access):** Allows I/O devices to transfer data directly to memory without continuously involving the CPU.
- **Parallel Processing (Multi-core CPUs):** Different cores process different camera feeds simultaneously.
- **Branch Prediction s Faster Clock Speed:** Reduce instruction delays and improve processing efficiency.

Conclusion

The lag in the smart city surveillance system is mainly caused by heavy data transfer between I/O devices, memory, and the CPU. A single-bus architecture creates a communication bottleneck because all components share one bus. Replacing it with a multi-bus architecture, along with improvements such as cache memory, DMA, pipelining, and multi-core processing, significantly reduces delays, improves data throughput, and enables faster and more accurate real-time threat detection.

2) Analysis of CPU Performance in a Real-Time Stock Trading Application

A real-time stock trading application executes millions of instructions every second. During high trading activity, the processor experiences a **high CPI (Cycles Per Instruction)** due to frequent memory accesses and branch instructions. This increases execution time and delays transaction processing.

Impact of the Fetch-Decode-Execute Cycle

The CPU processes every instruction through the following stages:

1. **Fetch**
    - The CPU fetches the instruction from memory.
    - Frequent memory accesses increase fetch time because the CPU must wait for data from RAM.
2. **Decode**
    - The control unit interprets the instruction.
    - Branch instructions require the CPU to determine the next instruction, adding extra delay.
3. **Execute**
    - The ALU performs arithmetic or logical operations.
    - If required data is not available due to memory delays, the CPU stalls.
4. **Write Back (Store)**
    - The execution result is stored back into registers or memory.
    - Heavy memory traffic slows this process.

Result: During high load, repeated memory accesses and branch instructions increase the execution time of each instruction, resulting in a high CPI and slower transaction processing.

Why CPI Becomes High

- Frequent memory access causes CPU waiting time.
- Cache misses force the processor to access slower main memory.
- Branch instructions may cause incorrect branch prediction.

- Pipeline stalls occur when the processor waits for data or branch outcomes.
- Overall instruction execution requires more clock cycles.

Methods to Reduce CPI

a) Increase Cache Memory

- Use larger and faster L1, L2, and L3 caches.
- Frequently used instructions and data remain in cache.
- Reduces memory access time.

b) Pipeline Execution

- Overlap fetch, decode, and execute stages.
- Multiple instructions execute simultaneously.
- Improves CPU throughput.

c) Branch Prediction

- Predicts the outcome of branch instructions.
- Reduces pipeline flushing and stalls.
- Lowers CPI.

d) Prefetching

- Fetches instructions and data before they are needed.
- Minimizes CPU waiting time.

e) Out-of-Order Execution

- Executes independent instructions while waiting for slower instructions.
- Keeps CPU resources busy.

f) Multi-Core Processing

- Different processor cores handle different transactions simultaneously.
- Increases parallel processing and system throughput.

Architectural Enhancements

Enhancement	Benefit
Cache Memory	Reduces memory access delay
Instruction Pipelining	Executes multiple instructions simultaneously
Branch Prediction	Reduces branch penalties
Out-of-Order Execution	Minimizes CPU idle time
Prefetching	Fetches data before it is required
Multi-Core CPU	Processes multiple trading requests in parallel
High-Speed Memory (DDR5, SRAM Cache)	Faster data transfer between memory and CPU

Program

START

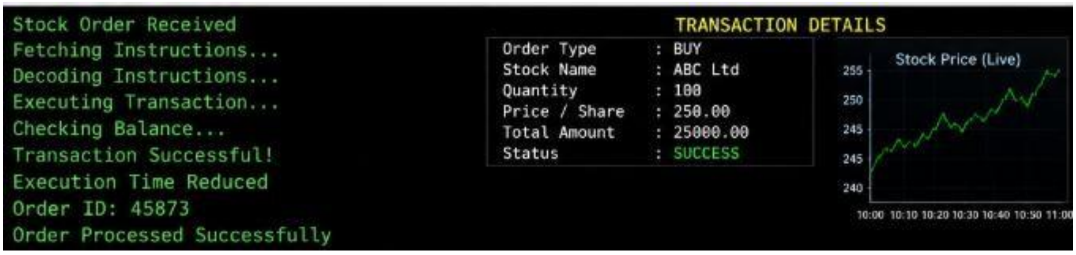
Read stock transaction

Fetch instruction
Decode instruction
Execute transaction

IF sufficient balance THEN
    Complete transaction
    Display "Transaction Successful"
ELSE
    Display "Transaction Failed"
ENDIF

STOP

OUTPUT



Overall Performance Improvement

- Lower **CPI (Cycles Per Instruction)**.
- Faster instruction execution.
- Reduced memory latency.
- Fewer pipeline stalls.
- Higher CPU utilization.
- Faster processing of stock transactions.
- Improved response time during peak trading hours.

Conclusion

In a real-time stock trading application, a high CPI is mainly caused by **frequent memory accesses and branch instructions**, which slow the **fetch-decode-execute cycle**. Performance can be significantly improved by using **cache memory, instruction pipelining, branch prediction, prefetching, out-of-order execution, multi-core processors, and faster memory**. These architectural enhancements reduce CPI, improve CPU efficiency, and ensure faster and more reliable transaction processing during periods of heavy trading.

3) Analysis of 8085 Addressing Modes in an Industrial Temperature Control Unit

An **8085 microprocessor-based temperature control system** receives temperature data from sensors (input devices) and controls actuators (such as heaters or cooling fans). If the program uses incorrect **addressing modes**, the microprocessor may read or process the wrong data, resulting in incorrect temperature control.

Influence of Addressing Modes on Data Access

The **8085 microprocessor** supports different addressing modes that determine how data is accessed.

a) Immediate Addressing Mode

- The data is given directly in the instruction.
- Example:

MVI A, 25H

- The accumulator receives **25H** directly.
- Used for fixed values such as threshold temperatures.

b) Register Addressing Mode

- Data is stored in CPU registers.
- Example:

MOV A, B

- Copies the contents of register **B** to **A**.
- Faster because no memory access is required.

c) Direct Addressing Mode

- The memory address is specified in the instruction.
- Example:

LDA 2500H

- Loads data from memory location **2500H** into the accumulator.

- Suitable for reading stored sensor values.

d) Register Indirect Addressing Mode

- The memory address is stored in a register pair (HL).
- Example:

MOV A, M

- Reads data from the memory location pointed to by the HL register pair.
- Useful when sensor data is stored in sequential memory locations.

e) Implied Addressing Mode

- The operand is implied by the instruction.
- Example:

CMA

- Complements the accumulator automatically.

Possible Causes of Incorrect Temperature Readings

- Incorrect memory address used in **Direct Addressing**, causing the wrong sensor value to be read.
- HL register pair not initialized correctly in **Indirect Addressing**.
- Wrong register selected in **Register Addressing**.
- Immediate values entered incorrectly.
- Programming errors causing sensor data to be overwritten.
- Noise or corrupted memory contents affecting stored values.

Using the 8085 Instruction Set Effectively

To ensure reliable operation, the following instructions should be used correctly:

Instruction	Purpose
LDA	Load sensor data from memory
STA	Store processed temperature value
MOV	Transfer data between registers
MVI	Load fixed threshold values
CMP	Compare temperature with reference value
JZ, JC, JNZ	Decision making based on comparison
IN	Read data from input ports (sensor interface)
OUT	Send control signals to actuators
HLT	Stop execution after completing tasks (if required)

Measures to Ensure Accurate and Reliable Operation

- Use the correct addressing mode for each operation.
- Initialize register pairs (HL, BC, DE) before indirect addressing.
- Verify memory addresses before accessing sensor data.
- Use comparison instructions (**CMP**) to validate readings.
- Store updated values correctly using **STA**.
- Use conditional branching (**JC, JZ, JNZ**) for proper control decisions.

Program

```
MVI A, 28H      ; Load temperature value
CPI 30H        ; Compare with threshold
JC COOL         ; If less than 30H
MVI A, 01H
OUT 01H         ; Turn ON Cooling Fan
HLT
COOL:
MVI A, 00H
OUT 01H         ; Fan OFF
HLT
```

OUTPUT

```
Temperature = 28°C
Threshold   = 30°C
-----------------------
Temperature Normal
Cooling Fan OFF
```

```
Temperature = 35°C
Threshold   = 30°C
-----------------------
High Temperature Detected
Cooling Fan ON
```

Conclusion

Addressing modes in the 8085 microprocessor determine how data is accessed from registers and memory. Incorrect use of these modes can cause the processor to read wrong temperature values, leading to improper control of industrial equipment. By selecting the appropriate addressing mode and using instructions such as LDA, STA, MOV, MVI, CMP, IN, OUT, and conditional jump instructions, the temperature control unit can process sensor data accurately and operate reliably.